10.1 Maps and Dictionaries

Python's **dict** class is arguably the most significant data structure in the language. It represents an abstraction known as a ***dictionary*** in which unique ***keys*** are mapped to associated ***values***. Because of the relationship they express between keys and values, dictionaries are commonly known as ***associative arrays*** or ***maps***. In this book, we use the term *dictionary* when specifically discussing Python's dict class, and the term *map* when discussing the more general notion of the abstract data type.

As a simple example, Figure 10.1 illustrates a map from the names of countries to their associated units of currency.

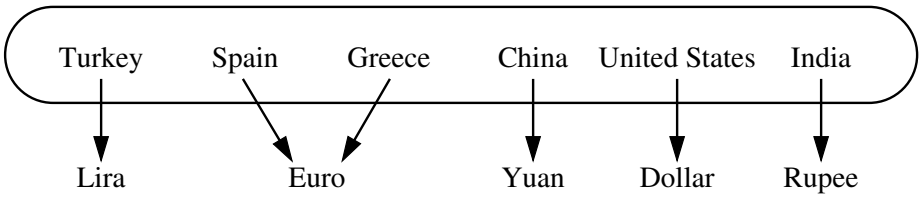


Figure 10.1: A map from countries (the keys) to their units of currency (the values).

We note that the keys (the country names) are assumed to be unique, but the values (the currency units) are not necessarily unique. For example, we note that Spain and Greece both use the euro for currency. Maps use an array-like syntax for indexing, such as currency['Greece'] to access a value associated with a given key or currency['Greece'] = 'Drachma' to remap it to a new value. Unlike a standard array, indices for a map need not be consecutive nor even numeric. Common applications of maps include the following.

- A university's information system relies on some form of a student ID as a key that is mapped to that student's associated record (such as the student's name, address, and course grades) serving as the value.
- The domain-name system (DNS) maps a host name, such as www.wiley.com, to an Internet-Protocol (IP) address, such as 208.215.179.146.
- A social media site typically relies on a (nonnumeric) username as a key that can be efficiently mapped to a particular user's associated information.
- A computer graphics system may map a color name, such as 'turquoise', to the triple of numbers that describes the color's RGB (red-green-blue) representation, such as (64,224,208).
- Python uses a dictionary to represent each namespace, mapping an identifying string, such as 'pi', to an associated object, such as 3.14159.

In this chapter and the next we demonstrate that a map may be implemented so that a search for a key, and its associated value, can be performed very efficiently, thereby supporting fast lookup in such applications.

10.1.1 The Map ADT

In this section, we introduce the ***map ADT***, and define its behaviors to be consistent with those of Python's built-in dict class. We begin by listing what we consider the most significant five behaviors of a map *M* as follows:

M[k]: Return the value v associated with key k in map M, if one exists; otherwise raise a KeyError. In Python, this is implemented with the special method __getitem__.

M[k] = v: Associate value v with key k in map M, replacing the existing value if the map already contains an item with key equal to k. In Python, this is implemented with the special method __setitem__.

del M[k]: Remove from map M the item with key equal to k; if M has no such item, then raise a KeyError. In Python, this is implemented with the special method __delitem__.

len(M): Return the number of items in map M. In Python, this is implemented with the special method __len__.

iter(M): The default iteration for a map generates a sequence of *keys* in the map. In Python, this is implemented with the special method __iter__, and it allows loops of the form, **for** k **in** M.

We have highlighted the above five behaviors because they demonstrate the core functionality of a map—namely, the ability to query, add, modify, or delete a key-value pair, and the ability to report all such pairs. For additional convenience, map *M* should also support the following behaviors:

k in M: Return True if the map contains an item with key k. In Python, this is implemented with the special __contains__ method.

M.get(k, d=None): Return M[k] if key k exists in the map; otherwise return default value d. This provides a form to query M[k] without risk of a KeyError.

M.setdefault(k, d): If key k exists in the map, simply return M[k]; if key k does not exist, set M[k] = d and return that value.

M.pop(k, d=None): Remove the item associated with key k from the map and return its associated value v. If key k is not in the map, return default value d (or raise KeyError if parameter d is None).

M.popitem( ): Remove an arbitrary key-value pair from the map, and return a (k,v) tuple representing the removed pair. If map is empty, raise a KeyError.

M.clear( ): Remove all key-value pairs from the map.

M.keys( ): Return a set-like view of all keys of M.

M.values( ): Return a set-like view of all values of M.

M.items( ): Return a set-like view of (k,v) tuples for all entries of M.

M.update(M2): Assign M[k] = v for every (k,v) pair in map M2.

M == M2: Return True if maps M and M2 have identical key-value associations.

M != M2: Return True if maps M and M2 do not have identical key-value associations.

Example 10.1: *In the following, we show the effect of a series of operations on an initially empty map storing items with integer keys and single-character values. We use the literal syntax for Python's dict class to describe the map contents.*

Operation	Return Value	Map
len(M)	0	{ }
M['K'] = 2	–	{'K': 2}
M['B'] = 4	–	{'K': 2, 'B': 4}
M['U'] = 2	–	{'K': 2, 'B': 4, 'U': 2}
M['V'] = 8	–	{'K': 2, 'B': 4, 'U': 2, 'V': 8}
M['K'] = 9	–	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M['B']	4	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M['X']	KeyError	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('F')	None	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('F', 5)	5	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('K', 5)	9	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
len(M)	4	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
del M['V']	–	{'K': 9, 'B': 4, 'U': 2}
M.pop('K')	9	{'B': 4, 'U': 2}
M.keys( )	'B', 'U'	{'B': 4, 'U': 2}
M.values( )	4, 2	{'B': 4, 'U': 2}
M.items( )	('B', 4), ('U', 2)	{'B': 4, 'U': 2}
M.setdefault('B', 1)	4	{'B': 4, 'U': 2}
M.setdefault('A', 1)	1	{'A': 1, 'B': 4, 'U': 2}
M.popitem( )	('B', 4)	{'A': 1, 'U': 2}

10.1.2 Application: Counting Word Frequencies

As a case study for using a map, consider the problem of counting the number of occurrences of words in a document. This is a standard task when performing a statistical analysis of a document, for example, when categorizing an email or news article. A map is an ideal data structure to use here, for we can use words as keys and word counts as values. We show such an application in Code Fragment 10.1.

We break apart the original document using a combination of file and string methods that results in a loop over a lowercased version of all whitespace separated pieces of the document. We omit all nonalphabetic characters so that parentheses, apostrophes, and other such punctuation are not considered part of a word.

In terms of map operations, we begin with an empty Python dictionary named freq. During the first phase of the algorithm, we execute the command

```
freq[word] = 1 + freq.get(word, 0)
```

for each word occurrence. We use the get method on the right-hand side because the current word might not exist in the dictionary; the default value of 0 is appropriate in that case.

During the second phase of the algorithm, after the full document has been processed, we examine the contents of the frequency map, looping over freq.items( ) to determine which word has the most occurrences.

```
1  freq = { }
2  for piece in open(filename).read( ).lower( ).split( ):
3    # only consider alphabetic characters within this piece
4    word = ''.join(c for c in piece if c.isalpha( ))
5    if word:    # require at least one alphabetic character
6      freq[word] = 1 + freq.get(word, 0)
7
8  max_word = ''
9  max_count = 0
10 for (w,c) in freq.items( ):        # (key, value) tuples represent (word, count)
11   if c > max_count:
12     max_word = w
13     max_count = c
14 print('The most frequent word is', max_word)
15 print('Its number of occurrences is', max_count)
```

Code Fragment 10.1: A program for counting word frequencies in a document, and reporting the most frequent word. We use Python's dict class for the map. We convert the input to lowercase and ignore any nonalphabetic characters.

10.1.3 Python's MutableMapping Abstract Base Class

Section 2.4.3 provides an introduction to the concept of an ***abstract base class*** and the role of such classes in Python's collections module. Methods that are declared to be abstract in such a base class must be implemented by concrete subclasses. However, an abstract base class may provide *concrete* implementation of other methods that depend upon use of the presumed abstract methods. (This is an example of the ***template method design pattern***.)

The collections module provides two abstract base classes that are relevant to our current discussion: the Mapping and MutableMapping classes. The Mapping class includes all nonmutating methods supported by Python's dict class, while the MutableMapping class extends that to include the mutating methods. What we define as the map ADT in Section 10.1.1 is akin to the MutableMapping abstract base class in Python's collections module.

The significance of these abstract base classes is that they provide a framework to assist in creating a user-defined map class. In particular, the MutableMapping class provides *concrete* implementations for all behaviors other than the first five outlined in Section 10.1.1: __getitem__, __setitem__, __delitem__, __len__, and __iter__. As we implement the map abstraction with various data structures, as long as we provide the five core behaviors, we can inherit all other derived behaviors by simply declaring MutableMapping as a parent class.

To better understand the MutableMapping class, we provide a few examples of how concrete behaviors can be derived from the five core abstractions. For example, the __contains__ method, supporting the syntax k in M, could be implemented by making a guarded attempt to retrieve self[k] to determine if the key exists.

```
def __contains__(self, k):
  try:
    self[k]                          # access via __getitem__ (ignore result)
    return True
  except KeyError:
    return False                     # attempt failed
```

A similar approach might be used to provide the logic of the setdefault method.

```
def setdefault(self, k, d):
  try:
    return self[k]                   # if __getitem__ succeeds, return value
  except KeyError:                   # otherwise:
    self[k] = d                      # set default value with __setitem__
    return d                         # and return that newly assigned value
```

We leave as exercises the implementations of the remaining concrete methods of the MutableMapping class.

10.1.4 Our MapBase Class

We will be providing many different implementations of the map ADT, in the remainder of this chapter and next, using a variety of data structures demonstrating a trade-off of advantages and disadvantages. Figure 10.2 provides a preview of those classes.

The MutableMapping abstract base class, from Python's collections module and discussed in the preceding pages, is a valuable tool when implementing a map. However, in the interest of greater code reuse, we define our own MapBase class, which is itself a subclass of the MutableMapping class. Our MapBase class provides additional support for the composition design pattern. This is a technique we introduced when implementing a priority queue (see Section 9.2.1) in order to group a key-value pair as a single instance for internal use.

More formally, our MapBase class is defined in Code Fragment 10.2, extending the existing MutableMapping abstract base class so that we inherit the many useful concrete methods that class provides. We then define a nonpublic nested _Item class, whose instances are able to store both a key and value. This nested class is reasonably similar in design to the _Item class that was defined within our PriorityQueueBase class in Section 9.2.1, except that for a map we provide support for both equality tests and comparisons, both of which rely on the item's key. The notion of equality is necessary for all of our map implementations, as a way to determine whether a key given as a parameter is equivalent to one that is already stored in the map. The notion of comparisons between keys, using the $<$ operator, will become relevant when we later introduce a ***sorted map ADT*** (Section 10.3).

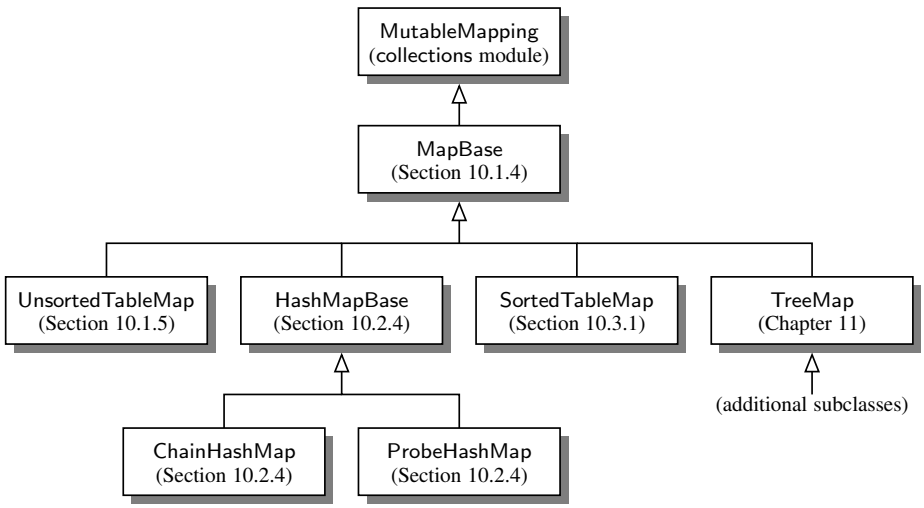


Figure 10.2: Our hierarchy of map types (with references to where they are defined).

```
1  class MapBase(MutableMapping):
2    """Our own abstract base class that includes a nonpublic _Item class."""
3
4    #------------------------------ nested _Item class ------------------------------
5    class _Item:
6      """Lightweight composite to store key-value pairs as map items."""
7      __slots__ = '_key', '_value'
8
9      def __init__(self, k, v):
10       self._key = k
11       self._value = v
12
13     def __eq__(self, other):
14       return self._key == other._key     # compare items based on their keys
15
16     def __ne__(self, other):
17       return not (self == other)         # opposite of __eq__
18
19     def __lt__(self, other):
20       return self._key < other._key      # compare items based on their keys
```

Code Fragment 10.2: Extending the MutableMapping abstract base class to provide a nonpublic _Item class for use in our various map implementations.

10.1.5 Simple Unsorted Map Implementation

We demonstrate the use of the MapBase class with a very simple concrete implementation of the map ADT. Code Fragment 10.3 presents an UnsortedTableMap class that relies on storing key-value pairs in arbitrary order within a Python list.

An empty table is initialized as self._table within the constructor for our map. When a new key is entered into the map, via line 22 of the __setitem__ method, we create a new instance of the nested _Item class, which is inherited from our MapBase class.

This list-based map implementation is simple, but it is not particularly efficient. Each of the fundamental methods, __getitem__, __setitem__, and __delitem__, relies on a for loop to scan the underlying list of items in search of a matching key. In a best-case scenario, such a match may be found near the beginning of the list, in which case the loop terminates; in the worst case, the entire list will be examined. Therefore, each of these methods runs in $O(n)$ time on a map with n items.

```
class UnsortedTableMap(MapBase):
  """Map implementation using an unordered list."""

  def __init__(self):
    """Create an empty map."""
    self._table = [ ]                                   # list of _Item's

  def __getitem__(self, k):
    """Return value associated with key k (raise KeyError if not found)."""
    for item in self._table:
      if k == item._key:
        return item._value
    raise KeyError('Key Error: ' + repr(k))

  def __setitem__(self, k, v):
    """Assign value v to key k, overwriting existing value if present."""
    for item in self._table:
      if k == item._key:                                # Found a match:
        item._value = v                                 # reassign value
        return                                          # and quit
    # did not find match for key
    self._table.append(self._Item(k,v))

  def __delitem__(self, k):
    """Remove item associated with key k (raise KeyError if not found)."""
    for j in range(len(self._table)):
      if k == self._table[j]._key:                      # Found a match:
        self._table.pop(j)                              # remove item
        return                                          # and quit
    raise KeyError('Key Error: ' + repr(k))

  def __len__(self):
    """Return number of items in the map."""
    return len(self._table)

  def __iter__(self):
    """Generate iteration of the map's keys."""
    for item in self._table:
      yield item._key                                   # yield the KEY
```

Code Fragment 10.3: An implementation of a map using a Python list as an unsorted table. Parent class MapBase is given in Code Fragment 10.2.